

CoreOps Week 5: Interview Prep

Q1. What is a PodDisruptionBudget, and why do we use it?

Answer: A PDB defines the minimum number of pods that **MUST** stay available during voluntary disruptions.

Used during:

- node drain
- rolling updates
- cluster maintenance

It ensures high availability by preventing all pods from being evicted simultaneously.

Q2. Difference between voluntary vs involuntary disruptions?

Answer:

- **Voluntary:** node drain, rolling update, admin-triggered action → PDB applies
- **Involuntary:** node crash, hardware failure → PDB ignored

Q3. What is a preStop hook? Why is it used?

Answer:

A lifecycle hook executed **before SIGTERM**.

Used to:

- gracefully drain connections
- stop background workers
- finish in-flight requests

Q4. What is the pod termination sequence in Kubernetes?

Answer:

1. SIGTERM sent
2. preStop hook executes
3. Readiness probe marks the pod Unready → removed from service endpoints
4. Grace period countdown
5. SIGKILL if still running

Q5. Why does graceful shutdown often fail in real systems?

Because:

- Developers ignore SIGTERM
- Misconfigured terminationGracePeriod
- Ingress sends traffic too fast
- Readiness probe does not fail early

SCENARIO QUESTIONS

Q6. During node drain, all pods of a service go down. What went wrong?

- No PDB
- PDB misconfigured
- Using maxUnavailable: 100%
- Pod labels do not match PDB selector

Q7. A pod terminates instantly. How do you debug?

Check: `kubectl describe pod`

`kubectl logs --previous`

Likely causes:

- `terminationGracePeriodSeconds = 0`
- SIGTERM ignored
- preStop hook fails immediately

Velero, Backups, Snapshots, Restore

Q1. What does Velero back up?

- Kubernetes API objects
- Persistent volume data (if using Restic or snapshot plugins)
- Namespace-level resources

Q2. What happens if you only back up Kubernetes objects without backing up PVCs?

App comes back but with empty or corrupted data.

Q4. How do you restore a namespace you accidentally deleted?

Use: `velero restore create --from-backup <name>`

Q5. You restored a namespace but pods stuck in CrashLoopBackOff. Why?

- PVC not restored
- PV reclaim policy = Delete
- Snapshot missing
- Secrets/config maps not included

Q6. Backup succeeded but restore failed — how do you debug?

Check: `velero restore describe <restore>`

`kubectl describe volumesnapshot`

`kubectl describe pvc`

Chaos Engineering Basics (Pod Kill, Latency Injection)

Q1. What is the goal of chaos engineering?

To validate how the system behaves under failure **before production fails**.

Q2. What is the difference between chaos testing and load testing?

- Chaos: break components
 - Load: stress system
- Chaos tests resilience, not performance.

Q3. What is the simplest chaos experiment?

Manually deleting a pod: `kubectl delete pod --force --grace-period=0`

Q4. You injected latency into a pod. What should you observe?

- Readiness & liveness probes
- Application timeout behavior
- CPU increases due to blocked calls
- Service-level impact

Q5. After chaos test, system did not recover. What now?

Check:

- Auto-scaling behavior
- PDB (prevented new pods)
- Network policies
- Resource limits throttling

Multi-Zone Architecture & Workload Spreading

Q1. Why do pods sometimes all land in the same zone?

Because scheduler chooses first available capacity.
Without topologySpreadConstraints or pod anti-affinity → no spreading.

Q2. What are topology spread constraints?

Rules that force pods to be evenly distributed across zones/nodes.

Q3. What happens if a whole zone goes down?

- Pods in that zone die
- Deployments auto-reschedule if enough nodes available
- PVCs may fail if using zonal storage

Q4. You have a deployment with 6 replicas but all land in zone-a. Fix it.

Add: topologySpreadConstraints:

- maxSkew: 1

topologyKey: topology.kubernetes.io/zone

Q5. Your database is zonal and node in that zone fails. What happens?

- PVC is stuck
- Pod cannot restart in another zone
- Application enters degraded state
Need regional storage.

Runbooks, Escalation Chains, Failover Drills

Q1. What makes a good runbook?

- Clear
- Step-by-step
- Minimal assumptions
- Copy-paste commands
- Verification steps

Q2. What is the difference between a playbook and a runbook?

- **Runbook:** step-by-step instructions
- **Playbook:** higher-level set of runbooks for incidents

Q3. What is RTO and RPO?

- **RTO:** Maximum allowable downtime
- **RPO:** Maximum allowable data loss

Q4. A service goes down at 3 AM. What is your escalation chain?

L1 checks → L2 triages → L3 specialist involved → L4 architect engaged.

Q5. Failover drill: traffic switched to v2 but clients still hitting v1. Why?

- Service selector still targets v1
- DNS caching
- Ingress rules not updated
- Old endpoints remain

Hands-On / Debugging Questions

Q1. A pod always restarts during rollout. What do you check?

- preStop hook taking too long
- readiness not failing fast
- terminationGracePeriod too small
- resource limits too small

Q2. Backup reports success but restore is empty. Why?

- Minio/S3 credentials wrong
- Restic repo not initialized
- PVC was excluded
- Backup only stored objects, not volumes

Q3. You drained a node but deployment went to 0 replicas. Why?

- PDB misconfigured
- No spare nodes in cluster
- maxUnavailable: 100% applied

Advanced SRE-Level Questions

Q1. What kind of failures should NEVER be tested with chaos engineering?

- Data corruption
- Permanent data deletion
- Cluster-wide outages without blast radius

Q2. How do you design DR for a multi-region architecture?

- Async replication
- DNS failover
- Global load balancing
- Region-level Velero backup
- Cloud provider snapshot replication

Q3. Can you have 0 data loss AND 0 downtime?

No, CAP theorem.

Trade-offs exist:

- RPO → data loss
- RTO → downtime

